bootcampproject 개발 정리

멘토링 플랫폼 · Spring Boot 4.1 / Java 17 / JPA / Thymeleaf · MySQL `bootcampproject`

작성 세션 기준: enum → entity → dto → repository → exception/config → service(User·Question·Answer·Appointment)

목차

1. 전체 구조와 계층
2. enum (Purpose, AppointmentStatus)
3. 엔티티 enum 반영
4. DTO 계층 (request / response)
5. repository 계층
6. PasswordEncoder 빈
7. exception 패키지
8. UserService
9. QuestionService
10. AnswerService
11. AppointmentService
12. 공통 규칙과 남은 작업

1. 전체 구조와 계층

요청은 다음 순서로 흐른다.

```
브라우저 폼
  → Controller        (세션에서 로그인 userIdx 꺼냄, @Valid 로 DTO 검증)
  → Service           (도메인 규칙 검증, 트랜잭션 경계, 엔티티 조립/변경)
  → Repository         (JPA 쿼리)
  → DB (MySQL)
반대 방향으로 Entity → Response DTO → 화면(Thymeleaf)
```

계층	역할	이 세션에서 한 일
entity	DB 테이블 매핑	enum 2개 반영 (purpose, status)
dto/request	폼 입력 + 검증 + `toEntity()` / `applyTo()`	13개
dto/response	출력 전용 + `static from(entity)`	7개
repository	쿼리 메서드	6개 인터페이스에 메서드 추가
exception	서비스가 "실패"를 표현하는 어휘	7개
config	빈 설정	PasswordEncoder
service	비즈니스 로직	User / Question / Answer / Appointment (4/6)

2. enum (Purpose, AppointmentStatus)

왜 String 대신 enum인가

String으로 두면 "이 필드는 정해진 몇 개 값 중 하나"라는 규칙을 코드가 표현할 수 없다. 오타(`"COUNCEL"`), 대소문자(`"counsel"`), 임의값이 컴파일·DB 어디에서도 안 걸리고 데이터가 오염된 뒤에야 발견된다.

enum이 주는 것	설명
값 집합이 타입으로 고정	잘못된 값은 작성 자체가 컴파일 에러
IDE 자동완성 · 리팩터링	`Purpose.` 치면 3개가 뜸, 이름 변경이 전체 전파
값에 동작 부착	`getLabel()`, `canTransitionTo()` 등을 값과 한곳에
switch 누락 감지	case 빠지면 컴파일러 경고
바인딩 단계 필터	폼에서 이상한 값 오면 400으로 걸러짐

> **트레이드오프** — 값이 DB/설정에서 동적으로 늘어나는 종류(사용자가 만드는 카테고리 등)엔 부적합. 그래서 `Question.category`, `Location.category`는 일부러 String으로 남김.

저장 방식

`@Enumerated(EnumType.STRING)` → 컬럼엔 `"COUNSEL"` 문자열 그대로 저장. DB 스키마·데이터는 String일 때와 동일하고 자바 코드에서만 타입 안전을 얻는다. `EnumType.ORDINAL`(0,1,2 숫자 저장)은 순서 바뀌면 데이터가 깨지므로 항상 `STRING`.

Purpose

```
@Getter
@RequiredArgsConstructor
public enum Purpose {
    COUNSEL("고민상담 목적"),
    SCHOOL("학교생활 목적"),
    CAREER("취업/학업 목적");
    private final String label;
}
```

값 집합은 `Sing_Up.html`의 `<option value>`와 일치.

AppointmentStatus

```
public enum AppointmentStatus {
    REQUESTED("예약 요청"), ACCEPTED("수락됨"), REJECTED("거절됨"),
    DONE("완료"), CANCELLED("취소됨");
    private final String label;

    public Set<AppointmentStatus> nextStates() {
        return switch (this) {
            case REQUESTED -> Set.of(ACCEPTED, REJECTED, CANCELLED);
            case ACCEPTED  -> Set.of(DONE, CANCELLED);
            case REJECTED, DONE, CANCELLED -> Set.of();
        };
    }
    public boolean canTransitionTo(AppointmentStatus target) {
        return nextStates().contains(target);
    }
}
```

상태 전이 규칙을 enum 안에 둠 → 서비스에서 `current.canTransitionTo(next)` 한 줄로 검증.

3. 엔티티 enum 반영

엔티티	변경
UserEntity.purpose	`String` → `Purpose`, `@Enumerated(EnumType.STRING)`, 빌더 파라미터 타입 변경
AppointmentEntity.status	`String` → `AppointmentStatus`, 동일
SignUpRequest.purpose	`String @NotBlank` → `Purpose @NotNull` (enum엔 `@NotBlank` 못 씀)

폼 `<select value="COUNSEL">` 값이 enum 이름과 같아 `@ModelAttribute` 바인딩 시 자동 변환. 미선택(빈 문자열)은 `null`로 변환돼 `@NotNull`이 잡는다. `@Enumerated(STRING)` + 기존 `VARCHAR` 컬럼이라 `ddl-auto: update`로 스키마 호환.

4. DTO 계층 (request / response)

설계 원칙

- 엔티티당 1개가 아니라 **화면·기능당 1개**. 회원가입과 프로필수정은 받는 필드가 달라 DTO도 별개.
- 전부 `record` (getter·생성자·equals 자동). 기존 프로젝트 컨벤션과 일치.
- **입력**은 `dto/request`에: Bean Validation + `toEntity(연관엔티티...)` 또는 `applyTo(엔티티)`.
- **출력**은 `dto/response`에: `static from(엔티티)` 팩토리. 엔티티 직접 반환 금지(LAZY 프록시·순환참조).
- 세션 유저(writer/mentee)는 DTO로 안 받고 컨트롤러에서 id로 주입.
- 연관 엔티티는 `mentorIdx`처럼 id(Long)만 받고 서비스에서 조회해 연결.

request (13개)

파일	매핑 메서드 / 특이사항
SignUpRequest	`toEntity(String encodedPassword)`, `@AssertTrue`로 비번확인 일치검증
LoginRequest	—
ProfileUpdateRequest	`applyTo(UserEntity)`
PasswordChangeRequest	current/new/newConfirm, `@AssertTrue isNewPasswordConfirmed()`. 평문 저장 안 함(서비스가 인코딩)
QuestionCreateRequest	`toEntity(UserEntity writer)`
QuestionUpdateRequest	`applyTo(QuestionEntity)`
AnswerCreateRequest	`toEntity(question, writer, parent)`, `parentAnswerIdx` nullable
AnswerUpdateRequest	`applyTo(AnswerEntity)`
AppointmentCreateRequest	`toEntity(mentee, mentor, location)` — REQUESTED 고정, `@Future`
AppointmentStatusUpdateRequest	`@NotNull AppointmentStatus status`
ReviewCreateRequest	`toEntity(...)` + `@AssertTrue` XOR (appointment/answer 중 정확히 하나)
LocationCreateRequest	`toEntity()`
LocationUpdateRequest	`applyTo(LocationEntity)`

response (7개) — 전부 `static from(...)`

파일	비고
UserResponse	민감 필드(비번) 제외. `Boolean.TRUE.equals(user.getIsVerified())` 로 null 안전 변환
QuestionListItemResponse	`from(question, long answerCount)` — 본문 없음
QuestionDetailResponse	`from(question, List<AnswerResponse>)`
AnswerResponse	`parentAnswerIdx` 로 계층 정보
AppointmentResponse	mentee/mentor/location 이름 평탄화
ReviewResponse	`appointmentIdx` / `answerIdx` 중 하나만 값
LocationResponse	—

예: UserResponse

```
public record UserResponse(
        Long idx, String name, String email, Purpose purpose,
        String university, String department, boolean verified, LocalDateTime createdAt
) {
    public static UserResponse from(UserEntity user) {
        return new UserResponse(
                user.getIdx(), user.getName(), user.getEmail(), user.getPurpose(),
                user.getUniversity(), user.getDepartment(),
                Boolean.TRUE.equals(user.getIsVerified()), user.getCreatedAt());
    }
}
```

`record` 라 필드 접근은 `userResponse.name()`. Thymeleaf에선 `${user.name}` 그대로 동작. 생성자 직접 안 쓰고 `from()` 으로만 생성.

toEntity vs applyTo

- **toEntity** — 신규 생성. DTO가 새 엔티티를 조립해 반환.
- **applyTo** — 수정. 조회해 온 **영속 상태** 엔티티에 필드를 덮어씀. `@Transactional` 안이면 dirty checking 으로 커밋 시 UPDATE 자동 발생 (save 호출 불필요).

5. repository 계층

기존엔 전부 빈 `JpaRepository` 상속만. 도메인별 쿼리 메서드 추가.

파생 쿼리 vs @Query

- **파생 쿼리** — 메서드 **이름**으로 쿼리 생성. 중첩 프로퍼티는 밑줄로 명시: `findByWriter_Idx` = `writer.idx` 조건.
- **@Query** — JPQL 직접. `join fetch`로 LAZY 연관을 미리 로딩할 때. 파라미터는 `@Param`으로 바인딩.
- 둘 다 컴파일 타임엔 검증 안 됨. **Spring 컨텍스트 부팅 시** 검증됨(그래서 테스트 실행이 곧 검증).

Repository	메서드
UserRepository	`existsByEmail`, `findByEmail`
QuestionRepository	`findByOrderByCreatedAtDesc(Pageable)`, `findByTitleContainingIgnoreCaseOrderByCreatedAtDesc`, `findByWriter_IdxOrderByCreatedAtDesc`, `countByWriter_Idx`, `@Query findDetailByIdx` (writer fetch join)
AnswerRepository	`@Query findByQuestionIdxWithWriter` (writer fetch join, 오래된순), `countByQuestion_Idx`, `countByWriter_Idx`
AppointmentRepository	`@Query findByMenteeIdxWithDetails`, `findByMentorIdxWithDetails` (mentee/mentor/location 3중 fetch join)
ReviewRepository	`findByMentor_IdxOrderByCreatedAtDesc`, `@Query findAverageRatingByMentorIdx` (→ `Double`), `existsByAppointment_Idx`, `existsByAnswer_IdxAndMentee_Idx`
LocationRepository	`findByAvailableTrue`, `findByCategory`

> **N+1 주의** — 목록에서 행마다 `countByQuestion_Idx`를 호출하면 쿼리가 N번 나감. 부트캠프 규모는 허용, 커지면 `GROUP BY` 카운트를 `@Query`로 한 번에.

> **테스트 픽스처 수정** — 전용 스키마로 새로 생성되며 `user_password`·`user_purpose` NOT NULL이 그대로 걸림. 4개 리포지토리 테스트의 `UserEntity.builder()`에 `.password("test-password")` + `.purpose(Purpose.CAREER)` 추가. `RelationshipMappingTest`는 `name`이 25자 → `user_name`(20자) 초과라 `.name(type)`으로 축소. 운영 코드는 안 건드림.

6. PasswordEncoder 빈

```
// build.gradle — 풀 시큐리티 아님, BCrypt만
implementation 'org.springframework.security:spring-security-crypto'
```

```
@Configuration
public class PasswordEncoderConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

서비스에서 `encode(raw)`로 저장하고 `matches(raw, encoded)`로 검증. 로그인 필터 체인은 두지 않음(세션 방식으로 컨트롤러가 처리 예정).

7. exception 패키지

하는 일

서비스 도중 잘못된 상황을 만나면 그 자리에서 `throw` → 나머지 로직 실행 안 하고 빠져나옴 → 스프링이 잡아 적절한 화면/응답으로 변환. **정상 경로만 코드에 남아 읽기 쉬워진다.**

왜 커스텀으로

- `null` / `boolean` 반환보다 안전 — 호출부가 체크를 깜빡해도 NPE 대신 명확한 예외에서 멈춤.
- 상황마다 다른 타입 → `@ExceptionHandler`에서 타입별로 상태코드·화면 분기 (404 / 403 / 400 ...).
- 공통 상위 `BusinessException` 하나로 몰아서 처리 가능.
- 메시지를 예외 클래스 안에 모음 (`InvalidStatusTransitionException(from, to)`).

클래스	언제
BusinessException	공통 상위 (RuntimeException)
ResourceNotFoundException	`findById(...).orElseThrow(...)` . `(name, id)` 생성자
DuplicateEmailException	가입 이메일 중복
InvalidPasswordException	로그인/비번변경 시 불일치
ForbiddenException	남의 글 수정·삭제 시도 (시큐리티 `AccessDeniedException`과 이름 안 겹치게)
InvalidStatusTransitionException	예약 상태 전이 위반. `(from, to)` → 라벨로 메시지
DuplicateReviewException	같은 대상 재리뷰

> 아직 `@ControllerAdvice`는 없음. 예외 클래스만 준비된 상태이고, 서비스에서 실제로 `throw` 하기 시작함.

8. UserService

```
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Transactional
    public UserResponse signup(SignUpRequest request) {
        if (userRepository.existsByEmail(request.email())) {
            throw new DuplicateEmailException(request.email());
        }
        String encodedPassword = passwordEncoder.encode(request.password());
        UserEntity saved = userRepository.save(request.toEntity(encodedPassword));
        return UserResponse.from(saved);
    }

    public UserResponse login(LoginRequest request) {
        UserEntity user = userRepository.findByEmail(request.email())
                .orElseThrow(() -> new InvalidPasswordException("이메일 또는 비밀번호가 올바르지 않습니
다."));
        if (!passwordEncoder.matches(request.password(), user.getPassword())) {
            throw new InvalidPasswordException("이메일 또는 비밀번호가 올바르지 않습니다.");
        }
        return UserResponse.from(user);
    }

    public UserResponse getProfile(Long userIdx) {
        return UserResponse.from(getUserOrThrow(userIdx));
    }

    @Transactional
    public UserResponse updateProfile(Long userIdx, ProfileUpdateRequest request) {
        UserEntity user = getUserOrThrow(userIdx);
        request.applyTo(user);
        return UserResponse.from(user);
    }

    @Transactional
    public void changePassword(Long userIdx, PasswordChangeRequest request) {
        UserEntity user = getUserOrThrow(userIdx);
        if (!passwordEncoder.matches(request.currentPassword(), user.getPassword())) {
            throw new InvalidPasswordException("현재 비밀번호가 일치하지 않습니다.");
        }
        user.setPassword(passwordEncoder.encode(request.newPassword()));
    }

    private UserEntity getUserOrThrow(Long userIdx) {
        return userRepository.findById(userIdx)
                .orElseThrow(() -> new ResourceNotFoundException("회원", userIdx));
```

```
    }
}
```

포인트

- 클래스에 `@Transactional(readOnly = true)`, 쓰기 메서드(`signup` / `updateProfile` / `changePassword`)에만 `@Transactional` 재선언해 오버라이드.
- `@RequiredArgsConstructor` — `final` 필드를 받는 생성자 자동 생성(생성자 주입).
- `login` 은 이메일이 없을 때와 비번이 틀릴 때 **같은 메시지** — 어느 쪽이 틀렸는지 노출하지 않음.
- `updateProfile` 은 `applyTo` 후 save 없음 → dirty checking.
- 세션 유저는 `userIdx` 파라미터로만 받음(컨트롤러가 세션에서 꺼냄).
- `getProfile` 은 통계(질문수·답변수·평균별점) 없이 `UserResponse` 만. 필요 시 별도 `ProfileStatsResponse` .

9. QuestionService

뼈대는 UserService와 동일. **추가되는 3가지**: ① 의존성 3개(Question·User·Answer Repository) ② 소유자 검증 ③ 페이징 + 목록/상세 조립.

```
@Service @RequiredArgsConstructor @Transactional(readOnly = true)
public class QuestionService {

    private final QuestionRepository questionRepository;
    private final UserRepository userRepository;
    private final AnswerRepository answerRepository;

    @Transactional
    public Long create(Long writerIdx, QuestionCreateRequest request) {
        UserEntity writer = userRepository.findById(writerIdx)
                .orElseThrow(() -> new ResourceNotFoundException("회원", writerIdx));
        QuestionEntity saved = questionRepository.save(request.toEntity(writer));
        return saved.getIdx();              // 컨트롤러가 redirect:/questions/{idx}
    }

    @Transactional
    public void update(Long questionIdx, Long loginUserIdx, QuestionUpdateRequest request) {
        QuestionEntity question = getQuestionOrThrow(questionIdx);
        checkOwner(question, loginUserIdx);
        request.applyTo(question);
    }

    @Transactional
    public void delete(Long questionIdx, Long loginUserIdx) {
        QuestionEntity question = getQuestionOrThrow(questionIdx);
        checkOwner(question, loginUserIdx);
        questionRepository.delete(question);
    }

    public Page<QuestionListItemResponse> getList(Pageable pageable) {
        return questionRepository.findByOrderByCreatedAtDesc(pageable)
                .map(q -> QuestionListItemResponse.from(
                        q, answerRepository.countByQuestion_Idx(q.getIdx())));
    }

    public Page<QuestionListItemResponse> search(String keyword, Pageable pageable) {
        return questionRepository
                .findByTitleContainingIgnoreCaseOrderByCreatedAtDesc(keyword, pageable)
                .map(q -> QuestionListItemResponse.from(
                        q, answerRepository.countByQuestion_Idx(q.getIdx())));
    }

    public QuestionDetailResponse getDetail(Long questionIdx) {
        QuestionEntity question = questionRepository.findDetailByIdx(questionIdx)
                .orElseThrow(() -> new ResourceNotFoundException("질문", questionIdx));
        List<AnswerResponse> answers = answerRepository.findByQuestionIdxWithWriter(questionIdx)
                .stream().map(AnswerResponse::from).toList();
        return QuestionDetailResponse.from(question, answers);
    }

    private QuestionEntity getQuestionOrThrow(Long questionIdx) {
        return questionRepository.findById(questionIdx)
                .orElseThrow(() -> new ResourceNotFoundException("질문", questionIdx));
    }

    private void checkOwner(QuestionEntity question, Long loginUserIdx) {
        if (!question.getWriter().getIdx().equals(loginUserIdx)) {
            throw new ForbiddenException("본인이 작성한 질문만 수정·삭제할 수 있습니다.");
```

```
        }
    }
}
```

포인트

- **소유자 검증** — UserService는 "내 userIdx로 내 것만" 건드려 검증 불필요했으나, 질문은 남의 글 수정 시도 가능 → `checkOwner`로 `writer.idx` vs 세션 유저 비교, 다르면 `ForbiddenException`.
- `Page.map(...)` — 페이지 메타데이터(전체 건수·페이지 번호)는 유지, 내용물만 DTO 변환.
- 행마다 `countByQuestion_Idx` → N+1. `from()` 안 `q.getWriter().getName()`도 LAZY 로딩 발생. `@Transactional`이라 세션 열려 동작. 부트캠프 규모는 OK.
- `getDetail`은 `findDetailByIdx`(writer fetch join) + 답변 별도 조회를 합침.
- `new QuestionListItemResponse.from(...)`은 틀림 — `from`은 static 팩토리라 `new` 없이 `QuestionListItemResponse.from(...)`.

10. AnswerService

뼈대 동일. 핵심 차이: `create`에서 조회가 최대 3번(질문·작성자·부모답변), 그리고 **부모답변 검증**.

```
@Service @RequiredArgsConstructor @Transactional(readOnly = true)
public class AnswerService {

    private final AnswerRepository answerRepository;
    private final QuestionRepository questionRepository;
    private final UserRepository userRepository;

    @Transactional
    public Long create(Long questionIdx, Long writerIdx, AnswerCreateRequest request) {
        QuestionEntity question = questionRepository.findById(questionIdx)
                .orElseThrow(() -> new ResourceNotFoundException("질문", questionIdx));
        UserEntity user = userRepository.findById(writerIdx)
                .orElseThrow(() -> new ResourceNotFoundException("회원", writerIdx));
        AnswerEntity parent = resolveParent(request.parentAnswerIdx(), question);
        AnswerEntity saved = answerRepository.save(request.toEntity(question, user, parent));
        return saved.getIdx();
    }

    @Transactional
    public void update(Long answerIdx, Long loginUserIdx, AnswerUpdateRequest request) {
        AnswerEntity answer = getAnswerOrThrow(answerIdx);
        checkOwner(answer, loginUserIdx);
        request.applyTo(answer);
    }

    @Transactional
    public void delete(Long answerIdx, Long loginUserIdx) {
        AnswerEntity answer = getAnswerOrThrow(answerIdx);
        checkOwner(answer, loginUserIdx);
        answerRepository.delete(answer);
    }

    public List<AnswerResponse> getByQuestion(Long questionIdx) {
        return answerRepository.findByQuestionIdxWithWriter(questionIdx)
                .stream().map(AnswerResponse::from).toList();
    }

    private AnswerEntity resolveParent(Long parentAnswerIdx, QuestionEntity question) {
        if (parentAnswerIdx == null) return null;             // 최상위 답변
        AnswerEntity parent = getAnswerOrThrow(parentAnswerIdx);
        if (!parent.getQuestion().getIdx().equals(question.getIdx())) {
            throw new BusinessException("대상 답변이 이 질문에 속해 있지 않습니다.");
        }
        return parent;
    }

    private AnswerEntity getAnswerOrThrow(Long answerIdx) {
        return answerRepository.findById(answerIdx)
                .orElseThrow(() -> new ResourceNotFoundException("답변", answerIdx));
    }

    private void checkOwner(AnswerEntity answer, Long loginUserIdx) {
        if (!answer.getWriter().getIdx().equals(loginUserIdx)) {
            throw new ForbiddenException("본인이 작성한 답변만 수정·삭제할 수 있습니다.");
        }
    }
}
```

resolveParent — 이 서비스의 핵심

`AnswerEntity`는 `parent` 필드로 자기 자신을 참조(대답변). `parentAnswerIdx`로 조회만 하면 **다른 질문의 답변**을 부모로 붙이는 이상한 데이터가 생길 수 있음 → 같은 질문 소속인지 검증. `parent.getQuestion()`은 LAZY지만 `create`가 `@Transactional`이라 자동 로딩. 딱 맞는 예외 타입이 없어 상위 `BusinessException`을 직접 throw.

getByQuestion

`QuestionService.getDetail` 내부와 같은 코드. 현재는 중복을 감수하고 서비스 독립 유지(서비스 간 호출 늘면 순환참조 위험).

11. AppointmentService

뼈대 동일(`@Service`, `@RequiredArgsConstructor`, `@Transactional(readOnly = true)`, 헬퍼). 의존성 3개: `AppointmentRepository`, `UserRepository`, `LocationRepository`. 핵심은 **상태 변경의 2단계 검증**.

```
@Service @RequiredArgsConstructor @Transactional(readOnly = true)
public class AppointmentService {

    private final AppointmentRepository appointmentRepository;
    private final UserRepository userRepository;
    private final LocationRepository locationRepository;

    @Transactional
    public Long create(Long menteeIdx, AppointmentCreateRequest request) {
        UserEntity mentee = userRepository.findById(menteeIdx)
                .orElseThrow(() -> new ResourceNotFoundException("회원", menteeIdx));
        UserEntity mentor = userRepository.findById(request.mentorIdx())
                .orElseThrow(() -> new ResourceNotFoundException("멘토", request.mentorIdx()));
        LocationEntity location = locationRepository.findById(request.locationIdx())
                .orElseThrow(() -> new ResourceNotFoundException("장소", request.locationIdx()));

        if (mentee.getIdx().equals(mentor.getIdx())) {
            throw new BusinessException("자기 자신에게 예약을 신청할 수 없습니다.");
        }
        if (!Boolean.TRUE.equals(location.getAvailable())) {
            throw new BusinessException("현재 예약할 수 없는 장소입니다.");
        }

        AppointmentEntity saved =
                appointmentRepository.save(request.toEntity(mentee, mentor, location));
        return saved.getIdx();
    }

    @Transactional
    public void changeStatus(Long appointmentIdx, Long loginUserIdx,
                             AppointmentStatusUpdateRequest request) {
        AppointmentEntity appointment = getAppointmentOrThrow(appointmentIdx);
        AppointmentStatus current = appointment.getStatus();
        AppointmentStatus next = request.status();

        if (!current.canTransitionTo(next)) {                       // 1단계: 전이 규칙
            throw new InvalidStatusTransitionException(current, next);
        }

        boolean isMentor = appointment.getMentor().getIdx().equals(loginUserIdx);
        boolean isMentee = appointment.getMentee().getIdx().equals(loginUserIdx);
        if (!isMentor && !isMentee) {                               // 2단계: 권한
            throw new ForbiddenException("이 예약의 당사자만 상태를 변경할 수 있습니다.");
        }
        switch (next) {
            case ACCEPTED, REJECTED, DONE -> {
                if (!isMentor) throw new ForbiddenException("수락 / 거절 / 완료는 멘토만 할 수 있습니
다.");
            }
            case CANCELLED -> { /* 멘토·멘티 모두 가능 */ }
            default -> throw new InvalidStatusTransitionException(current, next);
        }
        appointment.setStatus(next);
    }

    public List<AppointmentResponse> findAsMentee(Long userIdx) {
        return appointmentRepository.findByMenteeIdxWithDetails(userIdx)
                .stream().map(AppointmentResponse::from).toList();
```

```
    }
    public List<AppointmentResponse> findAsMentor(Long userIdx) {
        return appointmentRepository.findByMentorIdxWithDetails(userIdx)
                .stream().map(AppointmentResponse::from).toList();
    }

    private AppointmentEntity getAppointmentOrThrow(Long appointmentIdx) {
        return appointmentRepository.findById(appointmentIdx)
                .orElseThrow(() -> new ResourceNotFoundException("예약", appointmentIdx));
    }
}
```

create — 조회 3번 + 도메인 규칙 2개

- 멘티(세션 유저)·멘토·장소를 각각 조회.
- **자기 예약 금지**: `mentee.idx == mentor.idx` → `BusinessException`.
- **비활성 장소 금지**: `location.available`이 `true`가 아니면 `BusinessException`. `Boolean`이 null일 수 있어 `Boolean.TRUE.equals(...)`.
- `request.toEntity(...)`가 `AppointmentStatus.REQUESTED` 고정으로 조립.

changeStatus — 검증이 2단계

1. **전이가 규칙상 가능한가 (상태 머신)** — `current.canTransitionTo(next)`. enum이 정의한 그래프(`REQUESTED→ACCEPTED/REJECTED/CANCELLED`, `ACCEPTED→DONE/CANCELLED`, 나머지 종착)를 벗어나면 `InvalidStatusTransitionException`.
2. **그 전이를 이 사람이 할 권한이 있는가** — 제3자면 `ForbiddenException`. 수락·거절·완료는 **멘토**만, 취소는 **양쪽** 다. `default`는 방어용(1단계에서 `REQUESTED`는 이미 걸러짐).

> **왜 두 검증을 나누나** — "규칙상 불가능한 전이"(400)와 "가능하지만 네 권한 아님"(403)은 원인도 응답 코드도 다르다. 컨트롤러/Advice에서 다르게 처리.

findAsMentee / findAsMentor — 분리한 이유

멘티 화면은 "내가 신청한 예약", 멘토 화면은 "나에게 온 요청"이라 성격이 다름. 합쳐 반환하면 화면에서 다시 갈라야 함. 둘 다 `...WithDetails` 쿼리(mentee/mentor/location **3중 fetch join**)라 `from`이 LAZY 없이 이름을 꺼낸다.

12. 공통 규칙과 남은 작업

서비스 공통 규칙

- `@Service` + 클래스 `@Transactional(readOnly = true)`, 쓰기 메서드에만 `@Transactional`.
- 생성자 주입(`@RequiredArgsConstructor`).
- 서비스는 **엔티티를 받지 않고** DTO+id를 받음, **엔티티를 반환하지 않고** Response DTO 반환.
- `findById(...).orElseThrow(...)` 반복 → private `getXxxOrThrow` 헬퍼.
- 소유자 검증 → private `checkOwner` 헬퍼 (Question·Answer·Appointment 반복 패턴).

남은 작업

디렉토리	내용
service	ReviewService(XOR 대상·중복 체크), LocationService(CRUD)
controller	모든 `@PostMapping` — 현재 `HomeController`는 GET 정적 라우팅만. 폼 제출은 전부 405
세션/인증	로그인 유저 idx를 세션에 저장·조회하는 방식 결정
@ControllerAdvice	`BusinessException` → 화면/상태코드 매핑
템플릿	`th:object` / `th:field`/에러 표시 바인딩
IdFindRequest / PasswordResetRequest	Id_Find 화면 작업 시

현재 실행 상태

앱 부팅 + MySQL 연결 + 6개 테이블 생성/갱신 OK. GET으로 12개 템플릿 화면 이동 가능(정적). **폼 제출·쓰기 동작은 전부 불가**(컨트롤러 없음). 서비스(User·Question·Answer·Appointment)/DTO/repository/enum/exception은 컴파일·컨텍스트 검증 통과 상태이나 아직 컨트롤러가 호출하지 않음.